

Web Designing Basics

Understanding the Internet and How the Web Works

Learning Objectives

By the end of this lesson, you should be able to:

- Explain what the Internet is.
 - Understand what protocols are and why they are important.
 - Differentiate between a browser and a search engine.
 - Understand how websites work using the request-response model.
 - Identify common HTTP request methods.
-

1. What is the Internet?

The **Internet** is a global network of interconnected computers and devices that communicate with each other to share information.

Think of the Internet as a massive highway system that connects millions of computers around the world.

- Your computer is one house.
- A web server is another house.
- The Internet is the road connecting them.

Whenever you open a website, your computer communicates with another computer somewhere in the world through the Internet.

Example

When you visit:

```
https://www.google.com
```

Your computer sends a request to Google's server through the Internet, and Google's server sends the webpage back.

2. What is the World Wide Web (WWW)?

Many beginners think the Internet and the Web are the same thing, but they are different.

Internet	World Wide Web
Network of computers	Collection of websites
Connects devices	Delivers web pages

The **Web** is just one service that runs on the Internet.

Other Internet services include:

- Email
- Video calls
- Online gaming
- File sharing

3. What is a Protocol?

A **protocol** is a set of rules that computers follow when communicating with each other.

Example: Computers use protocols to exchange data correctly.

Without protocols, computers would not understand one another.

Common Internet Protocols

Protocol	Purpose
HTTP	Transfers web pages
HTTPS	Secure version of HTTP
FTP	Transfers files
SMTP	Sends emails
TCP/IP	Basic communication on the Internet
DNS	Converts domain names into IP addresses

Example

When you type:

```
https://www.example.com
```

Your browser communicates with the web server using **HTTPS**.

4. HTTP vs HTTPS

HTTP

HTTP stands for:

HyperText Transfer Protocol

It transfers web pages between browsers and servers.

Example:

```
http://example.com
```

HTTPS

HTTPS stands for:

HyperText Transfer Protocol Secure

It works exactly like HTTP but encrypts all communication.

Example:

```
https://example.com
```

HTTPS protects:

- Passwords

- Credit card information
- Personal information

Always prefer HTTPS websites.

5. What is a Browser?

A **browser** is software used to access and display websites.

A browser understands:

- HTML
- CSS
- JavaScript

It converts these files into the web pages you see.

Popular Browsers

- Google Chrome
 - Mozilla Firefox
 - Microsoft Edge
 - Safari
 - Opera
-

Responsibilities of a Browser

A browser:

- Sends requests to web servers
 - Fetches data
 - Downloads images
 - Displays the webpage
-

6. What is a Search Engine?

A **search engine** is a website that helps users find information on the Internet.

Instead of remembering every website address, users simply search using keywords.

Popular Search Engines

- Google
 - Bing
 - DuckDuckGo
 - Yahoo
-

Browser vs Search Engine

Browser	Search Engine
Software installed on your device	Website
Displays web pages	Finds web pages
Chrome	Google

Example

```
Chrome → Browser
Google → Search Engine
```

You use Chrome to open Google.

7. Client and Server

Every website works using two main components.

Client

The client is the user's device.

Examples:

- Laptop

- Mobile phone
- Tablet

The client requests information.

Server

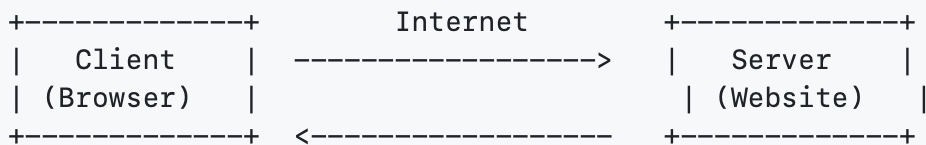
A server is a computer that stores websites and responds to requests.

The server sends:

- HTML
- CSS
- JavaScript
- Images
- Videos

back to the client.

Diagram



8. How Websites Work

Whenever you visit a website, a conversation happens between your browser and a server.

Step 1

User enters a URL.

`https://example.com`



Step 2

Browser sends an HTTP request.



Step 3

The request travels through the Internet.



Step 4

The server processes the request.



Step 5

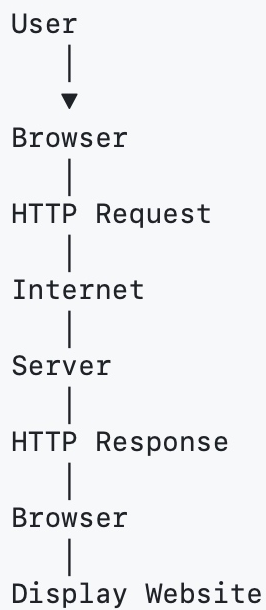
The server sends back an HTTP response.



Step 6

The browser displays the webpage.

Complete Flow



9. Request and Response

Communication on the web follows the **Request → Response** model.

Request

A request is sent by the browser asking for a resource.

Example HTTP Request Structure

An HTTP request consists of four main parts:

1. Request Line
2. Request Header Fields
3. Empty Line
4. Optional Message Body

Example

```
POST /login HTTP/1.1          ← Request Line
Host: www.example.com         ← Request Header
User-Agent: Mozilla/5.0       ← Request Header
Content-Type: application/json ← Request Header
Content-Length: 52            ← Request Header
Accept: application/json      ← Request Header

                                ← Empty Line

{                               ← Optional Message Body
  "username": "john_doe",
  "password": "secret123"
}
```

Breakdown

1. Request Line

The first line tells the server **what** the client wants.

```
POST /login HTTP/1.1
```

It consists of three parts:

Part	Description	Example
HTTP Method	Action to perform	POST
Request Target (Path)	Resource being requested	/login
HTTP Version	HTTP protocol version	HTTP/1.1

2. Request Header Fields

Headers provide additional information about the request.

```
Host: www.example.com
User-Agent: Mozilla/5.0
Content-Type: application/json
Content-Length: 52
Accept: application/json
```

Header	Purpose
Host	Specifies the target server
User-Agent	Identifies the client/browser
Content-Type	Indicates the format of the request body
Content-Length	Size of the request body in bytes
Accept	Specifies the response formats the client accepts

3. Empty Line

A blank line separates the **headers** from the **message body**.

This empty line tells the server:

"The headers are finished. The body begins next."

4. Optional Message Body

4. Optional Message Body

The message body contains the data sent to the server.

```
{
  "username": "john_doe",
  "password": "secret123"
}
```

The body is **optional** and is typically included with methods such as:

- POST
- PUT
- PATCH

It is usually **not** included with:

- GET
- DELETE (though technically allowed, it is uncommon)

Visual Structure

```
+-----+
| Request Line                               |
| POST /login HTTP/1.1                      |
+-----+
| Request Headers                           |
| Host: www.example.com                    |
| User-Agent: Mozilla/5.0                  |
| Content-Type: application/json           |
| Content-Length: 52                       |
| Accept: application/json                 |
+-----+
| Empty Line                               |
+-----+
| Optional Message Body                     |
| {                                         |
|   "username": "john_doe",               |
|   "password": "secret123"              |
| }                                         |
+-----+
```

Response

The server replies with the requested resource.

Example HTTP Response Structure

An HTTP response consists of four main parts:

- 1. Status Line
- 2. Response Header Fields
- 3. Empty Line
- 4. Optional Message Body

Example

```
HTTP/1.1 200 OK
Date: Tue, 28 Jul 2026 10:30:15 GMT
Server: Apache/2.4.62
Content-Type: application/json
Content-Length: 67
Connection: keep-alive

{
  "message": "Login successful",
  "token": "abc123xyz456"
}
```

← Status Line

← Response Header

← Response Header

← Response Header

← Response Header

← Response Header

← Empty Line

← Optional Message Body

Breakdown

1. Status Line

The first line tells the client **the result of the request**.

```
HTTP/1.1 200 OK
```

It consists of three parts:

Part	Description	Example
HTTP Version	Version of the HTTP protocol	HTTP/1.1
Status Code	Numerical result of the request	200
Reason Phrase	Human-readable description	OK

2. Response Header Fields

Headers provide additional information about the response.

```
Date: Tue, 28 Jul 2026 10:30:15 GMT
Server: Apache/2.4.62
Content-Type: application/json
Content-Length: 67
Connection: keep-alive
```

Header	Purpose
Date	Time when the response was generated
Server	Information about the web server
Content-Type	Format of the response body
Content-Length	Size of the response body in bytes
Connection	Indicates whether the TCP connection remains open

3. Empty Line

A blank line separates the **response headers** from the **message body**.

This empty line tells the client:

```
"The headers are finished. The response body begins next."
```

4. Optional Message Body

The message body contains the data returned by the server.

```
{
  "message": "Login successful",
  "token": "abc123xyz456"
}
```

The body is **optional**.

It commonly contains:

it commonly contains:

- HTML pages
- JSON data
- XML data
- Images
- Videos
- PDF files
- Error messages

Some responses, such as **204 No Content**, do not include a message body.

Visual Structure

```
+-----+
| Status Line                               |
| HTTP/1.1 200 OK                           |
+-----+
| Response Headers                          |
| Date: Tue, 28 Jul 2026 10:30:15 GMT        |
| Server: Apache/2.4.62                      |
| Content-Type: application/json             |
| Content-Length:67                          |
+-----+
| Empty Line                                |
+-----+
| Optional Message Body                      |
| {                                           |
|   "message": "Login successful",           |
|   "token": "abc123xyz456"                 |
| }                                           |
+-----+
```

10. HTTP Status Codes

Whenever a server responds, it sends a status code.

Some common ones are:

Code	Meaning
200	Success
201	Created
301	Moved Permanentlv

302	Temporary Redirect
400	Bad Request
401	Unauthorized
403	Forbidden
404	Not Found
500	Internal Server Error

11. HTTP Request Methods

Request methods tell the server **what action** should be performed.

GET

Used to retrieve information.

Example:

```
GET /products
```

Examples:

- Opening a webpage
 - Viewing products
 - Searching
-

POST

Used to send new data to the server.

Example:

```
POST /login
```

Examples:

- Login form
 - Registration form
 - Contact form
-

PUT

Used to replace an existing resource completely.

Example:

```
PUT /users/10
```

Example use:

Replace the complete user profile.

PATCH

Used to update only part of an existing resource.

Example:

```
PATCH /users/10
```

Example use:

Update only the email address.

DELETE

Used to remove data.

Example:

```
DELETE /users/10
```

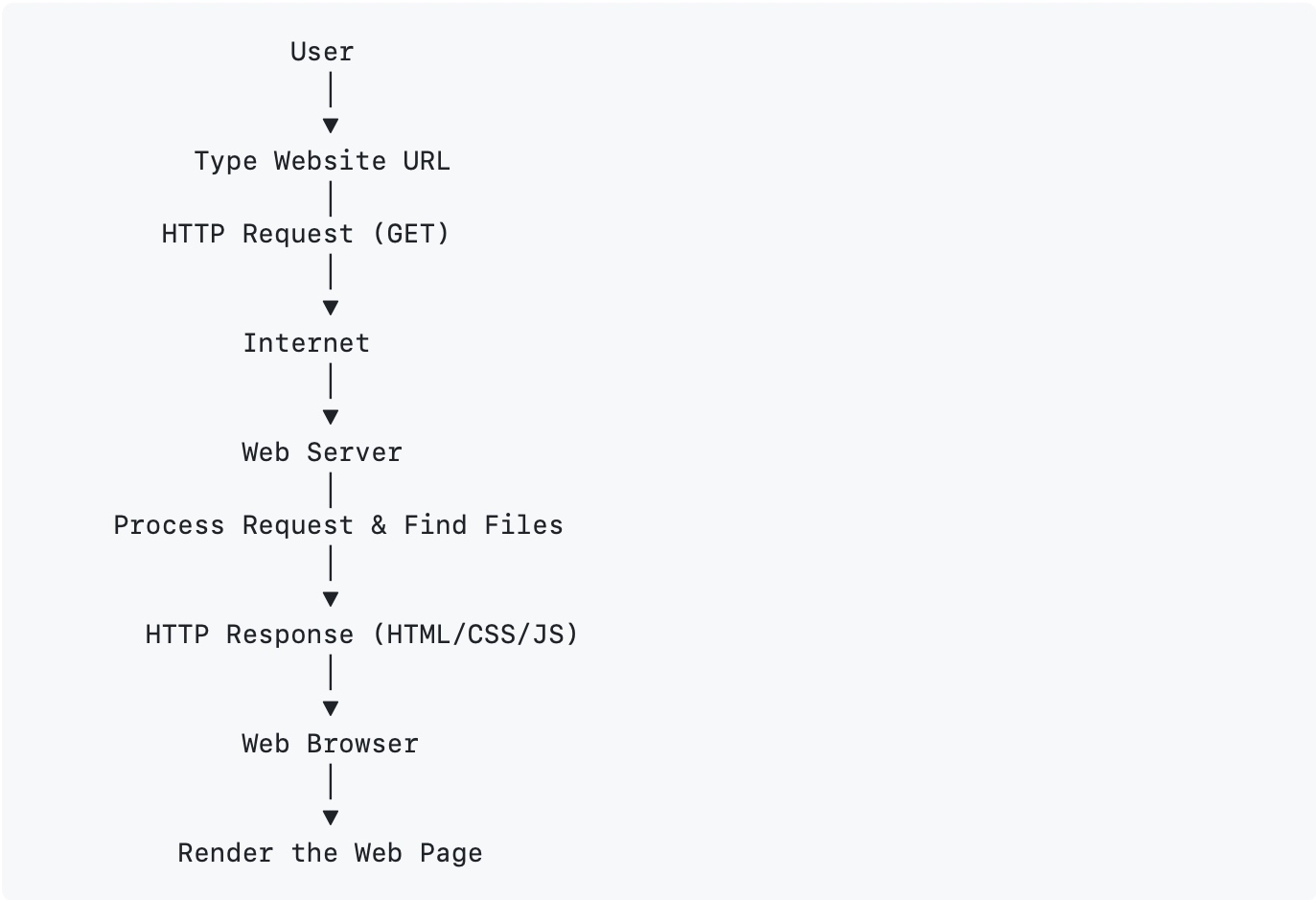
Example use:

Delete a user account.

Summary Table

Method	Purpose	Example
GET	Retrieve data	Open homepage
POST	Create new data	Submit registration
PUT	Replace existing data	Replace user profile
PATCH	Update part of data	Change password
DELETE	Remove data	Delete account

13. Putting Everything Together



Key Takeaways

- The **Internet** is a global network connecting computers.

- The **World Wide Web (WWW)** is a collection of websites that runs on the Internet.
- A **protocol** is a set of rules that allows computers to communicate.
- **HTTP** transfers web pages, while **HTTPS** does so securely using encryption.
- A **browser** displays websites and executes HTML, CSS, and JavaScript.
- A **search engine** helps users find websites and information.
- A **client** (browser) sends requests to a **server**, which stores website resources.
- Web communication follows the **Request → Response** model.
- **HTTP methods** define the action a client wants to perform:
 - **GET**: Retrieve data.
 - **POST**: Create new data.
 - **PUT**: Replace existing data.
 - **PATCH**: Update part of existing data.
 - **DELETE**: Remove data.